

# 高德 AI agent 前端开发

## 一面（技术基础） · 1-6

1. 简单介绍一下CSS的盒模型，以及 box-sizing 属性的区别。
2. 说一下 JavaScript 的事件循环机制，特别是宏任务和微任务的执行顺序。
3. 在 AI 聊天场景中，流式输出（Streaming）通常是怎么实现的？
4. 谈谈 Promise.all 和 Promise.allSettled 的区别，以及各自的应用场景。
5. 如何实现一个深拷贝函数？需要考虑哪些特殊情况？
6. 手写题：实现一个带有并发限制的异步调度器 Scheduler。

## 二面（深入原理与项目） · 1-8

1. AI 聊天界面中，当模型输出很长时，如何保证长列表的滚动性能？
2. 在开发 AI Agent 时，你是如何处理 Prompt 模板的？前端需要做哪些工程化工作？
3. 大模型返回的Markdown 包含代码块和公式，前端通常用什么方案渲染？
4. 谈谈 React 的 Fiber 架构，它为什么能提升复杂交互下的流畅度？
5. 什么是 Function Calling？前端在处理 Agent 的工具调用时需要注意什么？
6. 前端如何实现 RAG（检索增强生成）链路中的引用来源标注？
7. 谈谈 Web Worker 在 AI 前端应用中的使用场景。
8. 手写题：实现一个简单的状态机，模拟 AI Agent 的状态切换（Idle, Thinking, Calling\_Tool, Responding, Error）。

## 三面（架构综合） · 1-6

1. 如果让你从零设计一个 AI Agent 前端架构，你会考虑哪些核心模块？
2. 在高德地图这种业务场景下，AI Agent 如何与地图组件进行深度交互？
3. 如何衡量一个 AI 前端产品的用户体验？有哪些关键的技术指标？
4. 谈谈你对端侧模型（Local LLM）在前端应用前景的看法。
5. 聊聊你在过去一年中遇到的最难的 AI 相关前端问题，你是如何解决的？
6. 如果团队现在要从 Vue 迁移到 React，或者引入一套新的 AI 开发框架，你会如何推动？

## 一面（技术基础）

## 1. 简单介绍一下 CSS 的盒模型，以及 box-sizing 属性的区别。

**难度：★**

**考点：CSS 基础布局**

**答案要点：**

盒模型是浏览器对 HTML 元素进行布局的基础，规定了元素内容、内边距、边框和外边距的关系。

- 标准盒模型：width 仅包含 content，不包含 padding 和 border。
- 怪异盒模型（IE）：width 包含 content、padding 和 border。
- box-sizing: content-box：对应标准盒模型，是浏览器默认值。
- box-sizing: border-box：对应怪异盒模型，在响应式布局中更常用。

**常见坑：**

- 忽略了 padding 和 border 会撑开容器导致布局错位。
- 混淆了 margin 纵向重叠（Margin Collapsing）的特性。

**追问：**

- 如何触发 BFC？BFC 解决了什么问题？

## 2. 说一下 JavaScript 的事件循环机制，特别是宏任务和微任务的执行顺序。

**难度：★★**

**考点：JS 异步执行机制**

**答案要点：**

事件循环是 JS 处理异步操作的核心机制，通过任务队列协调同步代码与异步回调的执行。

- 执行栈清空：先执行同步代码，直到调用栈为空。
- 微任务优先：在每个宏任务执行完后，会清空当前所有的微任务队列（Promise.then、MutationObserver）。
- 宏任务调度：微任务清空后，从宏任务队列取出一个执行（setTimeout、setInterval、I/O）。
- 渲染时机：浏览器通常在微任务清空后、下一个宏任务开始前尝试进行 UI 渲染。

**常见坑：**

- 认为 setTimeout(fn, 0) 是立即执行。
- 混淆了 async/await 中 await 之后的代码属于微任务。

**追问：**

- Node.js 的事件循环和浏览器端有什么区别？

## 3. 在 AI 聊天场景中，流式输出（Streaming）通常是怎么实现的？

**难度：★★**

**考点：网络协议与流式传输**

### 答案要点：

流式输出主要利用 HTTP 的长连接特性，让服务端可以分批次向客户端推送文本片段。

- SSE (Server-Sent Events)：基于 HTTP 协议，单向推送，浏览器原生支持自动重连。
- WebSocket：全双工通信，适合需要频繁双向交互的场景，但协议开销相对较大。
- Fetch API + ReadableStream：目前主流做法，通过 fetch 获取 response.body 的 reader 逐块读取。
- Chunked Transfer Encoding：HTTP/1.1 的分块传输编码是实现流式的基础。

### 常见坑：

- 没考虑网络中断时的断开重连逻辑。
- 在前端处理流式数据时，直接拼接字符串导致大文本下的性能问题。

### 追问：

- SSE 相比于 WebSocket 有哪些优势和局限性？

## 4. 谈谈 Promise.all 和 Promise.allSettled 的区别，以及各自的应用场景。

### 难度：★

考点：异步并发控制

### 答案要点：

这两个方法都用于处理多个并发的 Promise，但对失败状态的处理逻辑完全不同。

- Promise.all：具有原子性，只要有一个失败就立即 reject，返回第一个失败的原因。
- Promise.allSettled：具有包容性，无论成功还是失败都会等待所有任务完成，返回每个任务的状态数组。
- all 场景：多个接口数据强依赖，必须全部成功才能渲染页面。
- allSettled 场景：多个独立任务，如批量上传文件或初始化多个互不影响的插件。

### 常见坑：

- 在 Promise.all 中没有写 catch 导致整个链路崩溃。
- 误以为 Promise.all 是按顺序执行的（其实是并发执行）。

### 追问：

- 如果想实现 Promise.any 的效果，你会怎么做？

## 5. 如何实现一个深拷贝函数？需要考虑哪些特殊情况？

### 难度：★★

考点：JS 数据类型处理

### 答案要点：

深拷贝需要递归遍历对象的所有属性，并创建全新的引用，以实现完全的解耦。

- 基础实现：使用递归处理 Object 和 Array。

- 循环引用：使用 WeakMap 存储已拷贝的对象，防止死循环。
- 特殊类型：处理 Date、RegExp、Map、Set 等特殊内置对象。
- 性能优化：对于超大对象，考虑按需拷贝或使用结构化克隆 API（structuredClone）。

#### 常见坑：

- 使用 JSON.parse(JSON.stringify()) 丢失函数、Symbol 和 undefined。
- 递归过深导致栈溢出。

#### 追问：

- structuredClone 有什么局限性？

## 6. 手写题：实现一个带有并发限制的异步调度器 Scheduler。

难度：★★★

考点：异步编程与队列控制

#### 答案要点：

该调度器需要控制同时进行的异步任务数量，多余的任务需进入等待队列。

- 核心逻辑：维护一个运行中的计数器和一个等待任务队列。
- 执行流程：每次 add 任务时检查计数器，未达上限则立即执行，否则入队。
- 任务完成：在任务的 finally 阶段从队列取出一个新任务执行，并递归此过程。

代码块

```
1  class Scheduler {
2    constructor(limit) {
3      this.limit = limit;
4      this.running = 0;
5      this.queue = [];
6    }
7    async add(fn) {
8      if (this.running >= this.limit) {
9        await new Promise(resolve => this.queue.push(resolve));
10     }
11     this.running++;
12     const result = await fn();
13     this.running--;
14     if (this.queue.length > 0) {
15       this.queue.shift()();
16     }
17     return result;
18   }
19 }
```

### 常见坑：

- 忘记在任务完成后释放计数器。
- 没能正确处理 Promise 的 resolve 触发时机。

## 二面（深入原理与项目）

### 1. AI 聊天界面中，当模型输出很长时，如何保证长列表的滚动性能？

难度：★★

考点：前端性能优化

答案要点：

针对高频更新的长列表，主要通过减少 DOM 节点数和优化渲染频率来提升性能。

- 虚拟列表（Virtual List）：只渲染可视区域内的消息节点，通过 padding 或 transform 模拟滚动条。
- 渲染分片：对于复杂的 Markdown 渲染，可以利用 requestIdleCallback 分片处理。
- 避免重排：固定消息气泡的宽度，使用 contain: layout 等 CSS 属性。
- 滚动锁定：实现用户向上滚动查看历史时停止自动滚动，新消息到来时显示提示。

常见坑：

- 虚拟列表在消息高度动态变化时，计算偏移量不准确导致抖动。
- 频繁的 setState 导致整个列表全量重新渲染。

追问：

- 如何处理消息中包含图片等异步加载内容导致的高度塌陷？

### 2. 在开发 AI Agent 时，你是如何处理 Prompt 模板的？前端需要做哪些工程化工作？

难度：★★

考点：AI 应用工程化

答案要点：

Prompt 工程在前端不仅是字符串拼接，更需要一套结构化和版本化的管理方案。

- 模板化管理：将 Prompt 定义为带占位符的配置，支持动态注入用户上下文（User Context）。
- 版本控制：对不同的 Prompt 版本进行灰度测试或 A/B Test。
- 安全过滤：在前端对用户输入进行预处理，防止简单的 Prompt 注入攻击。
- 结构化输出：配合大模型的 Function Calling，在 Prompt 中约束返回格式（如 JSON Schema）。

常见坑：

- 直接在代码里硬编码长文本 Prompt，导致难以维护。
- 忽略了 Token 长度限制，没有在前端做截断或预估。

**追问：**

- 如何在前端实现 Prompt 的自动优化或建议功能？

### 3. 大模型返回的 Markdown 包含代码块和公式，前端通常用什么方案渲染？

**难度：★★**

**考点：**内容解析与安全

**答案要点：**

渲染 AI 输出的内容需要兼顾解析效率、样式美化以及安全性。

- 解析引擎：常用 markdown-it 或 micromark，支持插件扩展（如代码高亮、数学公式）。
- 代码高亮：集成 prismjs 或 highlight.js，配合流式输出实现增量高亮。
- 公式渲染：使用 KaTeX 或 MathJax 处理 LaTeX 语法。
- 安全防御：必须经过 DOMPurify 进行 XSS 过滤，防止模型输出恶意脚本。

**常见坑：**

- 流式输出过程中，Markdown 标签未闭合导致渲染样式频繁闪烁。
- 渲染大型文档时阻塞主线程。

**追问：**

- 如何解决流式输出中代码块被截断导致的解析错误？

### 4. 谈谈 React 的 Fiber 架构，它为什么能提升复杂交互下的流畅度？

**难度：★★★**

**考点：**框架底层原理

**答案要点：**

Fiber 是对 React 核心算法的重构，将同步不可中断的更新变成了异步可中断的更新。

- 任务拆分：将渲染工作拆分为多个微小的 Work Unit（Fiber 节点）。
- 优先级调度：通过 Scheduler 给不同任务分配优先级（如用户输入高于数据请求）。
- 时间切片（Time Slicing）：在浏览器每一帧的空闲时间执行任务，避免长时间阻塞主线程。
- 双缓存技术：在内存中构建新的 Fiber 树，完成后一次性提交到真实 DOM。

**常见坑：**

- 误认为 Fiber 提升了单次渲染的速度（实际上可能变慢，但交互更响应）。
- 在 render 阶段执行了有副作用的操作。

**追问：**

- React 18 的 Concurrent Mode 是如何基于 Fiber 实现的？

## 5. 什么是 Function Calling? 前端在处理 Agent 的工具调用时需要注意什么?

难度: ★★★

考点: AI Agent 交互设计

答案要点:

Function Calling 允许模型根据需求调用外部工具, 前端负责展示调用过程并反馈执行结果。

- 状态展示: 需要设计专门的 UI 状态 (如 思考中、正在查询地图、调用成功)。
- 交互确认: 对于敏感操作 (如转账、删除), 前端必须拦截并弹出确认框。
- 错误处理: 当工具调用失败时, 需要将错误信息反馈给模型, 引导其重试或切换策略。
- 结果呈现: 将工具返回的结构化数据 (如 JSON) 转化为可视化的图表或卡片。

常见坑:

- 没处理好模型连续调用多个工具的串行/并行展示逻辑。
- 工具返回数据过大, 直接塞入 Context 导致 Token 溢出。

追问:

- 如何在 UI 上优雅地展示 Agent 的思考链 (CoT) ?

## 6. 前端如何实现 RAG (检索增强生成) 链路中的引用来源标注?

难度: ★★

考点: 数据关联与交互

答案要点:

引用标注是为了解决 AI 幻觉, 让用户可以追溯信息的真实来源。

- 数据结构: 后端返回的流式片段中需携带 source\_id 或 metadata 标识。
- 文本匹配: 在渲染层通过正则或偏移量匹配, 将引用标识 (如 [1]) 转化为可点击的锚点。
- 交互动效: 点击引用标识时, 侧边栏高亮显示对应的文档片段或 PDF 页面。
- 预取逻辑: 鼠标悬停在引用上时, 通过 Popover 预览来源摘要。

常见坑:

- 引用编号在流式输出过程中发生跳变。
- 原始文档过长, 前端定位锚点性能差。

追问:

- 如果来源是图片或视频, 你会如何设计展示逻辑?

## 7. 谈谈 Web Worker 在 AI 前端应用中的使用场景。

难度: ★★

考点: 多线程处理

答案要点:

Web Worker 可以将耗时计算移出主线程, 保证 AI 聊天界面的交互不卡顿。

- 文本预处理：对长文档进行分段（Chunking）或向量化（如果使用端侧模型）。
- 复杂解析：在大规模 Markdown 或 JSON 修复逻辑中使用。
- 离线模型：在浏览器端运行轻量级 Transformers.js 等模型时，必须在 Worker 中执行。
- 数据同步：处理 IndexedDB 中大量的向量数据检索。

#### 常见坑：

- 频繁的 PostMessage 导致序列化开销过大。
- 在 Worker 中无法直接操作 DOM 或访问某些全局 API。

#### 追问：

- 如何优化 Worker 与主线程之间的大数据传输？

## 8. 手写题：实现一个简单的状态机，模拟 AI Agent 的状态切换（Idle, Thinking, Calling\_Tool, Responding, Error）。

难度：★★

考点：设计模式与状态管理

答案要点：

状态机可以清晰地管理 Agent 复杂的交互逻辑，避免状态混乱。

- 定义状态：明确所有可能的合法状态。
- 定义动作：触发状态转移的事件。
- 转移逻辑：编写 transition 函数，判断当前状态是否允许执行目标动作。

代码块

```
1 class AgentStateMachine {
2   constructor() {
3     this.state = 'Idle';
4     this.transitions = {
5       Idle: { START: 'Thinking' },
6       Thinking: { CALL_TOOL: 'Calling_Tool', RESPONSE: 'Responding', ERROR:
7         'Error' },
8       Calling_Tool: { TOOL_DATA: 'Thinking', ERROR: 'Error' },
9       Responding: { FINISH: 'Idle', ERROR: 'Error' },
10      Error: { RESET: 'Idle' }
11    };
12  }
13  send(action) {
14    const nextState = this.transitions[this.state]?.[action];
15    if (nextState) {
16      this.state = nextState;
17      console.log(`State changed to: ${this.state}`);
18    } else {
19      console.warn(`Invalid action ${action} for state ${this.state}`);
20    }
21  }
22 }
```

```
19     }  
20     }  
21 }
```

#### 常见坑：

- 状态转移逻辑缺失，导致非法操作（如在 Error 状态下直接进入 Responding）。

## 三面（架构综合）

### 1. 如果让你从零设计一个 AI Agent 前端架构，你会考虑哪些核心模块？

难度：★★★

考点：系统架构设计

答案要点：

一个成熟的 Agent 前端架构需要具备高扩展性，能够适配不断演进的模型能力。

- 通信层：封装 SSE/WebSocket 逻辑，支持请求拦截、重试及多模型适配。
- 状态管理层：管理对话树（非线性对话）、插件状态、用户上下文及 Token 计数。
- 渲染引擎：可插拔的组件系统，支持文本、图表、地图、代码块等多种 Media Type。
- 插件/工具系统：定义标准化的工具接入协议，处理 Tool Call 的 UI 映射。
- 监控与评估：集成用户反馈（点赞/点踩）、性能监控（首字响应时间）及异常捕获。

常见坑：

- 架构设计过于耦合，导致更换大模型底层协议时需要重构整个 UI 层。
- 忽略了多轮对话中 Context 的清理策略。

### 2. 在高德地图这种业务场景下，AI Agent 如何与地图组件进行深度交互？

难度：★★★

考点：业务场景理解与技术结合

答案要点：

地图 Agent 的核心在于将自然语言指令转化为空间操作和地理数据展示。

- 指令转化：通过 Function Calling 将用户意图转化为地图 API 调用（如 setCenter, addMarkers）。
- 双向联动：地图上的交互（如点击兴趣点）能反向作为 Context 输入给 Agent。
- 空间上下文：在 Prompt 中注入当前视野内的 POI 信息，让 Agent 具备“视觉”感知。
- 异步渲染：Agent 搜索结果在地图上分批次渲染，并与左侧列表保持同步状态。

常见坑：

- 地图操作频繁导致性能瓶颈。
- Agent 输出的坐标信息与地图实际投影坐标系不统一。

### 3. 如何衡量一个 AI 前端产品的用户体验？有哪些关键的技术指标？

难度：★★★

考点：数据驱动与体验优化

答案要点：

除了传统的 Web 指标，AI 产品更关注响应速度和内容的确定性。

- TTFT (Time to First Token)：首字响应时间，直接影响用户感知的“快慢”。
- TPS (Tokens Per Second)：生成速度，反映了系统整体的吞吐能力。
- 请求成功率：特别是流式连接的稳定性。
- 交互转化率：如工具调用的点击率、引用来源的查看率。
- Evals 闭环：前端收集用户对回复的评价，作为模型微调或 Prompt 优化的依据。

常见坑：

- 只关注整体加载时间，忽略了流式输出过程中的卡顿感。

追问：

- 当后端响应慢时，前端可以采取哪些手段缓解用户的焦虑？

### 4. 谈谈你对端侧模型（Local LLM）在前端应用前景的看法。

难度：★★★

考点：技术视野

答案要点：

端侧模型是未来提升隐私性和降低成本的重要方向。

- 优势：极低的响应延迟、无需网络、数据隐私保护、节省服务端算力成本。
- 技术基础：WebGPU 的普及为浏览器端运行大模型提供了硬件加速可能（如 WebLLM）。
- 应用场景：简单的文本润色、敏感数据脱敏、离线代码补全、实时 UI 自动化控制。
- 挑战：模型体积与下载带宽的矛盾、移动端设备算力不均、显存占用问题。

常见坑：

- 盲目追求端侧运行，忽略了对用户设备性能的压榨可能导致浏览器崩溃。

### 5. 聊聊你在过去一年中遇到的最难的 AI 相关前端问题，你是如何解决的？

难度：★★★

考点：问题解决能力与复盘

答案要点：

此题没有标准答案，重点考察逻辑思维和技术深度。

- 问题背景：简洁描述业务场景和技术痛点（如：流式输出下的复杂表格渲染错乱）。
- 分析过程：使用了哪些工具排查（如：Chrome DevTools 性能分析、抓包分析协议）。
- 方案对比：当时考虑了哪几种方案，为什么选择了最终的那一个。
- 最终结果：量化的改进效果（如：渲染性能提升 50%，异常率降低到 0.1%）。

**常见坑：**

- 描述过于琐碎，没有体现出技术难度。
- 只有解决过程，没有总结沉淀出通用的方法论。

## 6. 如果团队现在要从 Vue 迁移到 React，或者引入一套新的 AI 开发框架，你会如何推动？

**难度：★★★**

**考点：工程领导力与技术选型**

**答案要点：**

技术迁移需要平衡业务稳定性与开发效率，采取循序渐进的策略。

- 必要性评估：分析新框架在处理 AI 复杂状态流、组件复用上的优势。
- 技术调研：建立 Demo 验证关键路径（如流式渲染、Hooks 与 AI 状态的结合）。
- 灰度方案：采用微前端或组件级替换的方式，先在非核心模块试点。
- 规范建设：制定新的代码规范、组件库适配方案及团队培训计划。

**常见坑：**

- 激进地进行全量重构，导致业务长时间停滞或引入大量回归 Bug。
- 忽略了团队成员的学习成本和心智负担。